→ How to write recursive code
→ Working of the recursive code
} Today

→ TC/SC recursive code } Wednesday's Session

Why Recursion?

→ Merge Sort / Quick Sort
→ Binary Trees / BST / BBST
→ Segment Trees
→ Heaps / Tries
→ Dynamic Programming
→ Backtracking
→ Graphs

} All these topics use Recursion

What is Recursion? → function calling itself

→ Recursion calls the same function but with a smaller input size.

e.g.

$sum(N) = 1 + 2 + 3 + 4 + \dots + (N-1) + N$

$sum(N) = sum(N-1) + N$

problem — smaller version of same problem → subproblem

Recursion is solving a problem using a subproblem.

Here, you have to believe.

You have to believe that Recursion will work

Have faith

Steps to write Recursive code

1) Assumption:- You decide what your recursive function should do, and you assume that it will do it. (Have faith in your func.).

2) Main logic:- Solving original problem using subproblem.

3) Base condition:- When should the code stop.

Given N, return sum of N natural numbers.

```
int sum(N){
    Base condition:- if(N==1) return 1;
    Assumption:- Given N, calculate sum of all numbers from
                 [1-N] & return it.
                 sum(3) = (1+2+3) = 6 ; sum(4) = 10
    Main logic:- return sum(N-1) + N
}
```

* Factorial

3! = 3 * 2 * 1 = 6

4! = 4 * 3 * 2 * 1 = 24

N! = N * (N-1) * (N-2) ... * 2 * 1 → fact(N-1)

for a given N, calculate N! and return it.

```
int fact(int N) {
    Base condition:- fact(1) = fact(0) * 1   (1 → 0)
                     if(N==1) return 1;

    Assumption:- Give N, will calculate N! and return it.
                 fact(3) = 6 ; fact(4) = 24

    Main logic :- return fact(N-1) * N
}
```

Fibonacci Series

	1st	2nd	3rd	4	5	6	7	(N-2)th 8	(N-1)th 9	Nth 10
fib() =	1	1	2	3	5	8	13	21	34	55

```
int fib(N) {
    |
    |
    |
    |
    |
    |
    |
}
```

fib(1) = fib(0) + fib(-1) → invalid
fib(2) = fib(1) + fib(0) → invalid
fib(3) = fib(2) + fib(1)

→ Need to write base condition for both of them.

Base condition:- `if (N==1 || N==2) return 1;`

Assumption:- Given N, calculate & return Nth fibonacci Number

Main logic:- return fib(N-1) + fib(N-2)

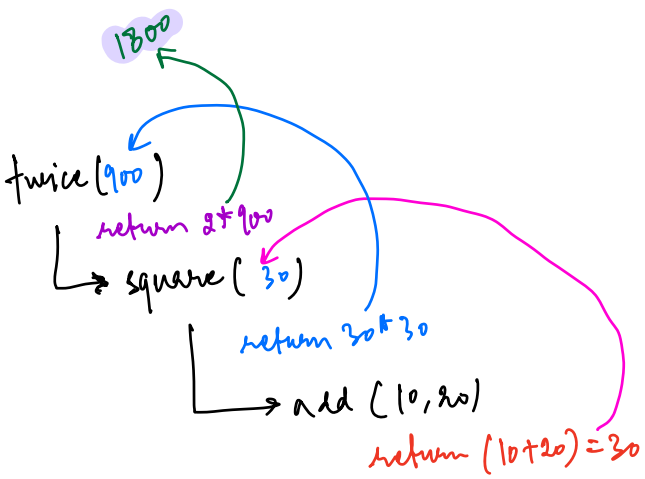
```
int fib(int N) {
    if(N==1 || N==2) return 1;
    return fib(N-1) + fib(N-2);
}
```

e.g.

```
int add (N,M){
    return N+M;
}
int square (N){
    return N*N;
}
int twice (N){
    return 2*N;
}
```

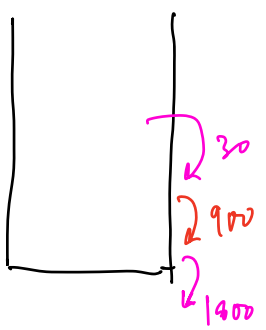
```
main(){
    int x = add(10,20);   → x=30
    int y = square(x);    → y=900
    int z = twice(y);     → z=1800
    print(z);             → 1800
}
```

```
main(){
    print(twice(square(add(10,20))));
}
```

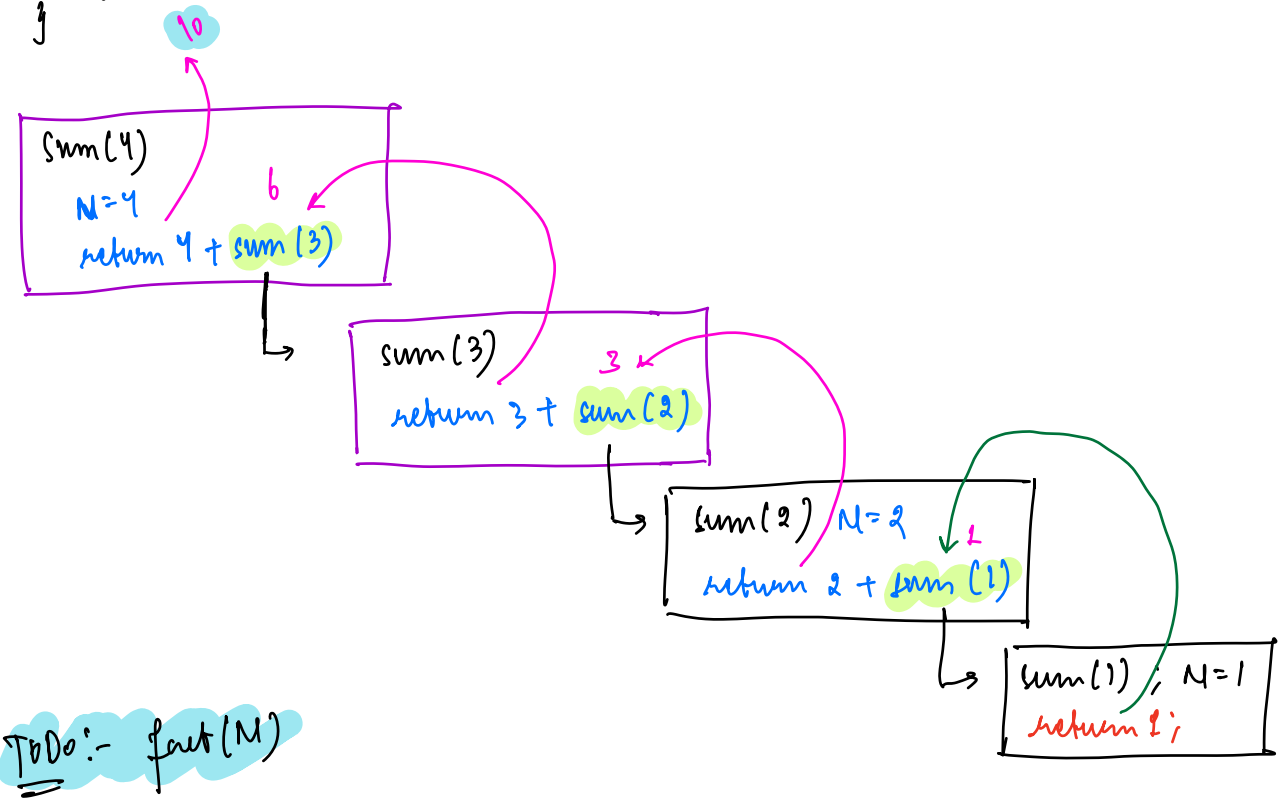


→ they are stored in memory

→ Stack is a DS used to store this



```
int sum(N) {
    if(N==1) return 1;
    return sum(N-1) + N;
}
```



TODO:- fact(N)

. — N

→ print 1, 2, 3, 4, 5, - - - N-1, N

```
void printInc(N)
```

Base cond.:- `if(N==1){print(1); return;}`

→ returns control back to the function which called it.

Ass:- Given N, print all number from [1~N]

printInc(3) → 1, 2, 3

Main logic:- printIn(4) → 1, 2, 3, 4

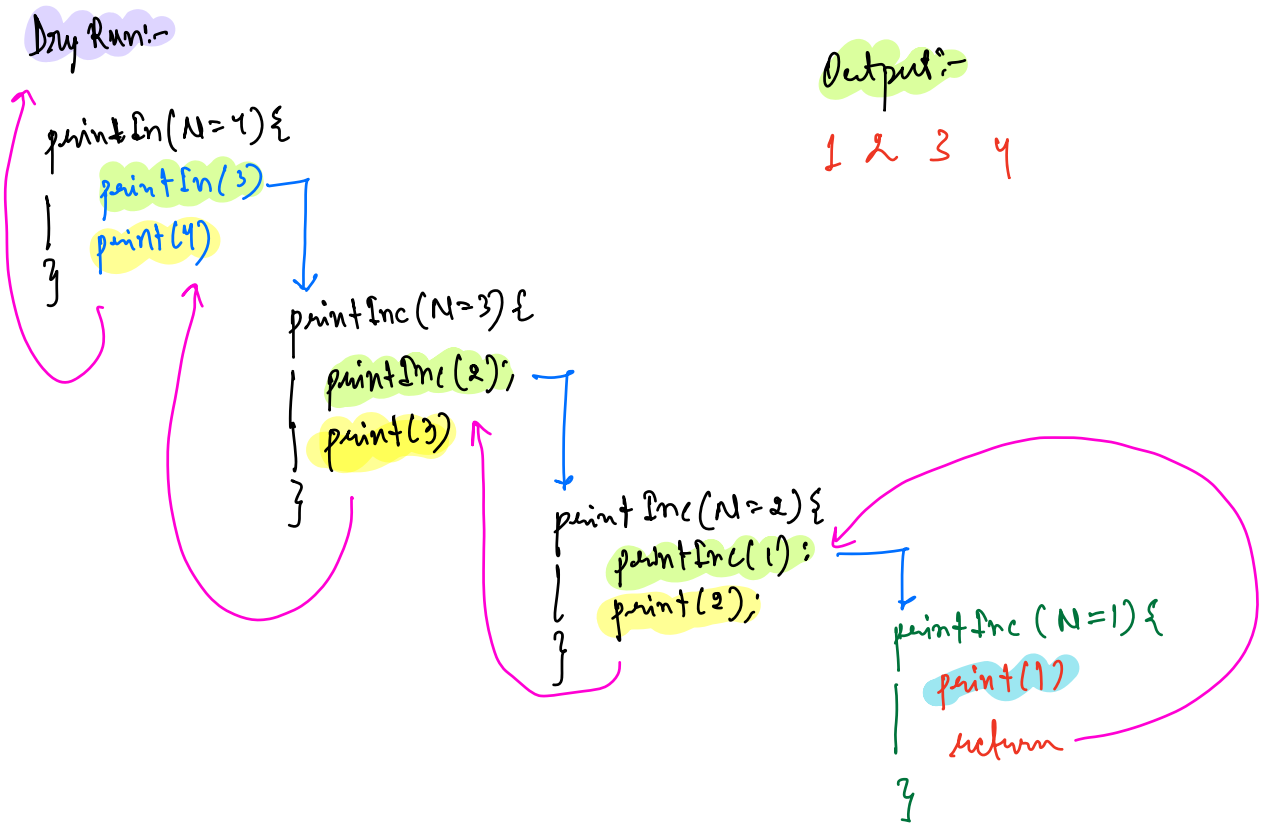
printInc(5) → 1, 2, 3, 4, 5

printInc(N-1) → This will print all number from [1, N-1]

print(N)

```
}
```

```
void printInc(N)
{
    if(N==1){
        print(1);
        return;
    }
    printInc(N-1);
    printf(N);
}
```


Dry Run:-
printFn(N=4){
printFn(3)
print(4)
}
printFnc(N=3){
printFnc(2);
print(3)
}
printFnc(N=2){
printFnc(1);
print(2);
}
printFnc(N=1){
print(1)
return
}
Output:-
1 2 3 4

↱ N, N-1, N-2 - - - 2, 3, 1

```
printDec(N)
{
    if(N==1) print(1); return;
    print(N)
    printDec(N-1);
}
```

Q Given a string, check palindrome or not?

e.g.1 A B C B A

e.g.2 a c e g a → Not palindrome

e.g.3 l o r r o l

start index, end index

```
bool palindrome(str, s, e){
    Assumption:- Given str, it will check whether substring
                 from [s,e] is palindrome or not and
                 return T/F.

    Base condition:-
        if(s >= e){
            return True;
        }

    Main logic:-
        if(str[s] == str[e]){
            if(palindrome(str, s+1, e-1))
                return True;
            }
            else{
                return false;
            }
        }
        else{
            return false;
        }
}
```